

Summary

This topic synthesizes technical insights from an instructional session regarding algorithmic recurrence relations. It details the methodologies for determining time complexity, focusing on the iteration (back substitution) method and the comprehensive application of the Extended Master Theorem.

The primary objective of analyzing recurrence relations is to derive the time complexity of recursive algorithms. This analysis shifts from exact mathematical counts to asymptotic notations (Big O, Theta), focusing on the growth rate of functions. Two primary techniques are established:

- 1. The Iteration Method: A manual process of expanding recursive calls to identify a generalizable pattern, which is then solved using base cases and mathematical series.
- 2. The Extended Master Theorem: A robust framework for solving recurrences of the form $T(n) = aT(n/b) + \Theta(n^k \log^p n)$. This method categorizes relations into three primary cases based on the comparison between recursive branching $(\log_b a)$ and the work performed at each level (k) .

Critical takeaways include the necessity of mathematical rigor over coding knowledge when analyzing algorithms and the importance of eliminating lower-order terms during asymptotic analysis.

The Iteration Method (Back Substitution)

The iteration method involves substituting the recursive definition into itself repeatedly until a pattern emerges. This pattern is then generalized to k steps and evaluated at the point where the recursion terminates (the base case).

Foundational Examples and Derivations

The following table summarizes the recurrence relations solved via the iteration method:

Recurrence Relation	Characteristics	Asymptotic Complexity
$(T(n) = T(n-1) + c)$	Problem size reduces by 1; constant work per step.	$(\Theta(n))$
$(T(n) = T(n-1) + cn)$	Problem size reduces by 1; work is proportional to current size.	$(\Theta(n^2))$

$(T(n) = T(n/2) + c)$	Problem size halves; constant work per step.	$(O(\log n))$
$(T(n) = 2T(n/2) + n)$	Two sub-problems of half size; work is linear.	$(O(n \log n))$
$(T(n) = 2T(n/2) + c)$	Two sub-problems of half size; constant work per step.	$(O(n))$

Detailed Analysis of Recursive Patterns

- **Linear Reductions:** In $(T(n) = T(n-1) + cn)$, the work follows an arithmetic progression $(1 + 2 + 3... + n)$, resulting in a sum formula of $(n(n+1)/2)$, which simplifies to $(O(n^2))$.
- **Logarithmic Reductions:** In $(T(n) = T(n/2) + c)$, the problem reaches the base case in approximately $(\log n)$ steps. Since constant work (c) is performed at each step, the complexity is $(O(\log n))$.
- **Geometric Expansions:** In $(T(n) = 2T(n/2) + c)$, the expansion creates a Geometric Progression (GP). Summing the work across k levels where $(n = 2^k)$ results in a complexity of $(O(n))$.

The Extended Master Theorem

The Extended Master Theorem is presented as a superior alternative to standard versions, as it accommodates logarithmic terms and provides a more streamlined decision-making process for objective analysis.

Standard Form

The theorem applies to recurrences following this structure:

$$\begin{bmatrix} T(n) = aT(n/b) + \theta(n^k \log^p n) \end{bmatrix}$$

Where:

- $(a \geq 1)$: Number of sub-problems.
- $(b > 1)$: Factor by which problem size is reduced.
- $(k \geq 0)$: Power of n in the non-recursive component.
- (p) : A real number representing the power of the logarithmic term.

Decision Logic: The Three Cases

Analysis begins by comparing $(\log_b a)$ with (k) .

Case 1: Recursive Heavy

- Condition: ($\log_b a > k$)
- Result: ($T(n) = \theta(n^{\log_b a})$)

Case 2: Balanced Work

- Condition: ($\log_b a = k$)
- The work at each level is roughly equal to the recursive branching. The result depends on the power of the logarithm (p):

Sub-case	Condition	Result
e		
Case 2a	($p > -1$)	($T(n) = \theta(n^k \log^{p+1} n)$)
Case 2b	($p = -1$)	($T(n) = \theta(n^k \log(\log n))$)
Case 2c	($p < -1$)	($T(n) = \theta(n^k)$)

- Example: ($T(n) = 2T(n/2) + n \log n$). Here, ($a = 2$), ($b = 2$), ($k = 1$), ($p = 1$). Since ($\log_2 2 = 1$) and ($k = 1$), and ($p > -1$), the complexity is ($\theta(n^1 \log^2 n)$).

Case 3: Work-at-Root Heavy

- Condition: ($\log_b a < k$)
- The non-recursive work performed at the current level dominates.

Sub-case	Condition	Result
e		
Case 3a	($p \geq 0$)	($T(n) = \theta(n^k \log^p n)$)
Case 3b	($p < 0$)	($T(n) = O(n^k)$)

- Example: ($T(n) = T(n/2) + n \log n$). Here, ($\log_2 1 = 0$), while ($k = 1$). Since ($0 < 1$), Case 3 applies. With ($p = 1$), the result is ($\theta(n \log n)$).

Key Methodological Insights

Asymptotic vs. Exact Analysis

The analysis prioritizes asymptotic behavior over exact step counting. Constants (c) and lower-order terms (e.g., (n) in the presence of ($n \log n$)) are discarded to focus on the highest growth rate.

Mathematical Requirements

Solving recurrences is fundamentally a mathematical exercise rather than a programming one. Proficiency in the following areas is required:

- Logarithms: Specifically properties like $(\log_b a)$ and the identity $(n = 2^k \implies k = \log_2 n)$.
- Series: Arithmetic and Geometric Progressions (GP) are the backbone of the iteration method.
- Comparison: The ability to compare exponential and logarithmic growth rates is essential for selecting the correct Master Theorem case.

Limitations of Master Theorem

The Master Theorem is not a universal solver. It is inapplicable to recurrences that do not fit the $(aT(n/b))$ structure, such as those with linear reductions $((n-1))$ or those where $(b \nleq 1)$. In such instances, the iteration method remains the primary analytical tool.